

VÙNG 2: CẤU TRÚC DỮ LIỆU & LÝ THUYẾT TÍNH TOÁN

Mô hình hóa Máy trạng thái hữu hạn (DFA) và Biến đổi trạng thái hàm thuần túy cho F# Core

Mục lục tài liệu

1. Hình thức hóa Máy Trạng thái Hữu hạn (Deterministic Finite Automaton - DFA)	1
1.1. Không gian trạng thái Q	1
1.2. Không gian sự kiện đầu vào Σ	2
1.3. Không gian hành động đầu ra $\mathcal{A}$	2
2. Biến đổi Trạng thái Hàm Thuần túy (Pure Functional State Transition)	2
2.1. Bảng chuyển trạng thái mở rộng	2
3. Cấu trúc Dữ liệu Bộ đệm (Immutable Buffer & Rollback)	3
3.1. Lịch sử trạng thái và Cơ chế Backtracking (Undo / Rollback)	3
3.2. Nhận diện Ranh giới Từ (Word Boundary Detection)	3
4. Cấu trúc Lưu trữ Từ điển và Bảng Tra cứu (Lookup Data Structures)	3
4.1. Mảng Tra cứu Trực tiếp (Direct Flat Array Lookup) cho Âm học	3
4.2. Cây Tiền tố (Radix Tree / Trie) cho Bảng Gõ tắt và Từ điển Hợp lệ	3

1. Hình thức hóa Máy Trạng thái Hữu hạn (Deterministic Finite Automaton - DFA)

Trong kiến trúc lõi của bộ gõ, việc nhận diện và cấu tạo âm tiết được mô hình hóa toán học bằng một bộ ngũ thành tố DFA:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Trong đó:

- Q : Tập hữu hạn các trạng thái của Engine.
- Σ : Bảng chữ cái đầu vào (tập hợp tất cả các mã phím gõ từ bàn phím và cờ điều khiển Modifier).
- $\delta : Q \times \Sigma \rightarrow Q \times \mathcal{A}$: Hàm chuyển trạng thái kèm theo hành động xuất (Action output).
- $q_0 \in Q$: Trạng thái khởi tạo mặc định (Idle).
- $F \subset Q$: Tập hợp các trạng thái kết thúc/hợp lệ (Terminal valid states).

1.1. Không gian trạng thái Q

Khác với mô hình hướng đối tượng dùng biến toàn cục khả biến, F# Core mô hình hóa không gian trạng thái thành một tập hợp các trạng thái loại trừ lẫn nhau (Discriminated Unions):

1. q_{Idle} : Trạng thái rỗng. Bộ đệm rỗng, con trỏ văn bản không chứa âm tiết đang soạn thảo dở dang.
2. $q_{\text{Buffering}}$: Đang gom cụm phụ âm đầu $\mathcal{O}$ hoặc các ký tự khởi đầu nhưng chưa xác lập nguyên âm chính.
3. $q_{\text{Composing}}$: Đang xây dựng cấu trúc âm tiết hợp lệ $S = (O, M, N, C, T)$.

4. $q_{\text{Passthrough}}$: Chuỗi ký tự nhập vào vi phạm luật ngữ âm tiếng Việt (từ tiếng Anh, chuỗi code, hoặc định danh biến). Mọi phím bấm tiếp theo trong từ này được chuyển tiếp trực tiếp vào ứng dụng mà không can thiệp biến đổi dấu.

1.2. Không gian sự kiện đầu vào Σ

Mỗi phần tử $\sigma \in \Sigma$ đại diện cho một sự kiện gõ phím hoàn chỉnh:

$$\sigma = (k, m)$$

- $k \in [0, 255]$: Mã ký tự hoặc Virtual Key code.
- $m \in \{\text{None, Shift, Ctrl, Alt, Meta}\}$: Mặt nạ phím bổ trợ.

1.3. Không gian hành động đầu ra $\mathcal{A}$

Sau mỗi bước chuyển trạng thái, Engine phát ra một hành vi cụ thể cho Adapter phía ngoài (C++ Fcidx5 hoặc C# TSF):

$$\mathcal{A} = \begin{cases} \text{Pass} & \text{Cho phép phím đi thẳng vào ứng dụng} \\ \text{Consume} & \text{Nuốt phím, không chuyển tiếp cho ứng dụng} \\ \text{Commit}(s) & \text{Gửi chuỗi hoàn chỉnh } s \text{ vào văn bản} \\ \text{Replace}(b, s) & \text{Gửi } b \text{ phím Backspace, sau đó gửi chuỗi mới } s \end{cases}$$

2. Biến đổi Trạng thái Hàm Thuần túy (Pure Functional State Transition)

Trọng tâm của F# Core là triệt tiêu hoàn toàn hiệu ứng lề (Side-effects). Toàn bộ chu trình gõ được gói gọn trong một hàm thuần túy:

$$\text{step} : Q \times \Sigma \rightarrow Q \times \mathcal{A}$$

2.1. Bảng chuyển trạng thái mở rộng

Trạng thái hiện tại (q)	Sự kiện vào (σ)	Trạng thái kế tiếp (q')	Hành động (α)	Giải thích nghiệp vụ
Idle	Ký tự alpha c	Buffering([c])	Pass	Ký tự đầu tiên của từ, lưu vào bộ đệm và cho hiển thị bình thường.
Buffering(buf)	Ký tự biến đổi dấu c	Composing(...)	Replace(b, s)	Khớp thành công luật gõ, lùi con trỏ và đề ký tự mang dấu.
Composing(s)	Ký tự vi phạm Phonotactics	Passthrough(...)	Pass	Phát hiện vi phạm cấu trúc tiếng Việt, hủy chế độ gõ dấu.
Composing(s)	Dấu cách (Space) / Dấu câu	Idle	Pass	Kết thúc âm tiết, đưa bộ đệm về rỗng để đón từ tiếp theo.
*	Tổ hợp Ctrl+Key / Alt+Key	Idle	Pass	Gãy mạch gõ do phím tắt hệ thống, reset toàn bộ bộ đệm.

Bảng 1: Ma trận chuyển trạng thái cốt lõi của Engine

3. Cấu trúc Dữ liệu Bộ đệm (Immutable Buffer & Rollback)

Engine không sử dụng mảng cố định có thể ghi đè như trong C++, mà sử dụng cấu trúc danh sách liên kết bất biến (Immutable Persistent List) để phục vụ quay lui.

3.1. Lịch sử trạng thái và Cơ chế Backtracking (Undo / Rollback)

Để giải quyết bài toán người dùng muốn sửa phím hoặc bấm phím lặp để thoát dấu (ví dụ: gõ s thành á, gõ thêm s nữa thành as), Engine duy trì một ngăn xếp trạng thái (State Stack):

$$\mathcal{H} = [q_n, q_{n-1}, \dots, q_0]$$

Mỗi khi nhận một phím σ , trạng thái mới q_{n+1} được sinh ra và đẩy lên đầu ngăn xếp. Khi người dùng nhấn phím Backspace:

$$\text{pop} : \mathcal{H} \rightarrow (q_n, \mathcal{H}')$$

Thuật toán quay lui chỉ việc lấy lại trạng thái q_{n-1} trước đó và hoàn trả chuỗi tương ứng.

3.2. Nhận diện Ranh giới Từ (Word Boundary Detection)

Một từ tiếng Việt được định nghĩa toán học là một chuỗi ký tự liên tục nằm giữa hai biên phân cách (Delimiters).

Tập ký tự ngắt từ được định nghĩa:

$$\mathcal{D} = \mathcal{W} \cup \mathcal{P} \cup \mathcal{S}$$

- $\mathcal{W}$: Tập ký tự khoảng trắng (Space, Tab, Newline).
- $\mathcal{P}$: Tập dấu câu (., ,, ;, :, !, ?, ", ', (,) ...).
- $\mathcal{S}$: Tập toán tử và ký tự đặc biệt (+, -, *, /, =, <, >, @, #, \$...).

Khi ký tự $\sigma_k \in \mathcal{D}$, hàm chuyển trạng thái xác định biên kết thúc:

$$\delta(q, (\sigma_k, m)) = (q_{\text{Idle}}, \text{Pass})$$

4. Cấu trúc Lưu trữ Từ điển và Bảng Tra cứu (Lookup Data Structures)

Để kiểm tra tính hợp lệ của âm tiết và hỗ trợ mở rộng từ viết tắt (Macro), F# Core sử dụng hai cấu trúc dữ liệu chính:

4.1. Mảng Tra cứu Trực tiếp (Direct Flat Array Lookup) cho Âm học

Đối với các kiểm tra ngữ âm học ở mức độ phần tử đơn (kiểm tra nguyên âm, phụ âm, bảng mã Unicode), F# Core sử dụng các mảng băm phân giải trực tiếp theo mã ASCII/Codepoint:

- Độ phức tạp truy vấn: $O(1)$
- Chi phí bộ nhớ: Tối ưu trực tiếp trong bộ nhớ đệm L1/L2 của CPU, không cấp phát heap.

4.2. Cây Tiên tố (Radix Tree / Trie) cho Bảng Gõ tắt và Từ điển Hợp lệ

Đối với tập từ điển âm tiết hợp lệ $\mathcal{S}_{\text{valid}}$ (khoảng hơn 6.000 âm tiết tiếng Việt) hoặc bảng gõ tắt mở rộng:

- Độ phức tạp tìm kiếm: $O(k)$ với k là độ dài chuỗi ký tự đang gõ ($k \leq 7$).
- Được biểu diễn bằng cấu trúc dữ liệu bất biến dạng cây:

Cấu trúc Cây Tiên tố (Trie Node)

Node = (IsTerminal : bool, Children : Map < char, Node >)